

Medii de proiectare și programare

2025-2026

Curs 8

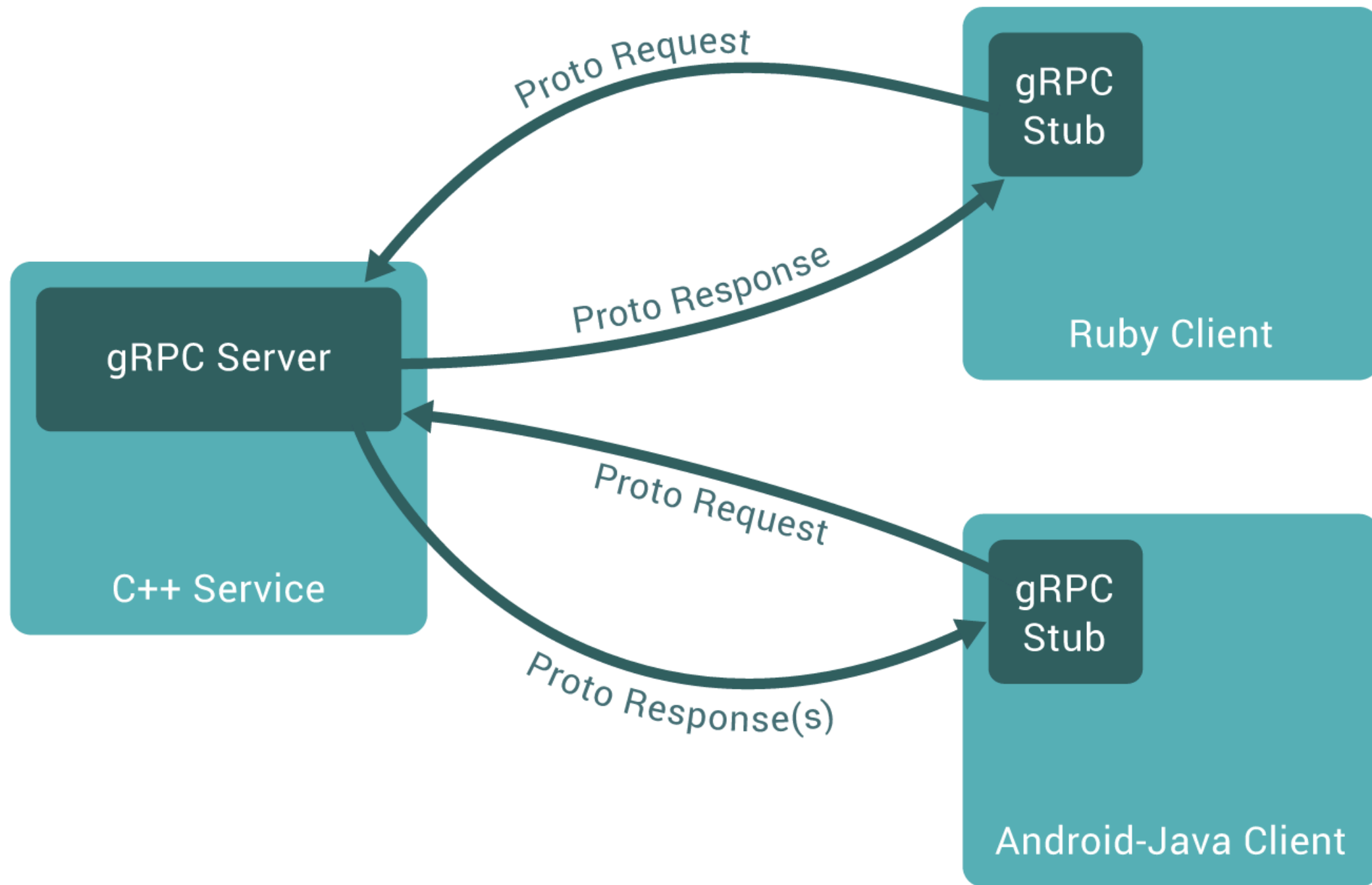
Continut

- Remote Procedure Call
 - gRPC
 - Thrift
- ORM
 - Hibernate (Java)
 - Entity Framework (C#)

gRPC

- Cu **gRPC** o aplicație client poate apela metode la distanță ca și cum ar fi metode ale unui obiect local.
- Facilitează crearea aplicațiilor distribuite și a serviciilor.
- gRPC folosește ideea definirii unui serviciu (interfețe) care specifică metodele ce pot fi apelate la distanță, împreună cu parametrii și tipul returnat al acestora.
- În aplicația server există un obiect remote care implementează interfața, iar serverul gRPC gestionează cererile clienților.
- Clientul (aplicația client) are un stub (proxy) care oferă aceleași metode ca și obiectul remote.
- Clienții și serverele gRPC pot fi scrise și pot rula în diferite limbaje de programare: Java, C#, C++, Python, Go, etc.

gRPC



gRPC - Definirea unui serviciu

- Implicit, gRPC folosește Protocol Buffers ca și limbaj de definire a tipurilor de mesaje și a serviciilor (IDL).
- Se pot folosi alte IDL-uri.

```
service HelloService {  
    rpc SayHello (HelloRequest) returns (HelloResponse);  
}
```

```
message HelloRequest {  
    string greeting = 1;  
}
```

```
message HelloResponse {  
    string reply = 1;  
}
```

gRPC - Definirea unui serviciu

- gRPC permite specificarea a 4 tipuri de apeluri de metode la distanță:
 - *unare* - clientul trimite o singură cerere la server și primește un singur răspuns de la server (apeluri de funcții normale).

```
rpc SayHello (HelloRequest) returns (HelloResponse) {}
```

- *server streaming* - clientul trimite o cerere la server și primește un *stream* cu ajutorul căruia poate citi o secvență de răspunsuri de la server. Clientul citește răspunsurile până când nu mai sunt mesaje pe stream.

```
rpc LotsOfReplies (HelloRequest) returns (stream HelloResponse) {}
```

- *client streaming* - clientul scrie o secvență de mesaje și le trimite la server folosind streamul furnizat. După ce clientul a terminat scrierea mesajelor, așteaptă ca serverul să le citească și să trimită un singur răspuns.

```
rpc LotsOfGreetings (stream HelloRequest) returns (HelloResponse) {}
```

gRPC - Definirea unui serviciu

- gRPC permite specificarea a 4 tipuri de apeluri de metode la distanță:
 - *Bidirectional streaming* - atât serverul cât și clientul trimit o secvență de mesaje folosind un stream bidirecțional (read-write). Streamurile (read-write) funcționează independent; clienții și serverul pot citi și scrie în ordinea dorită de ei.

Exemple:

-serverul poate aștepta până primește toate mesajele de la client înainte de a trimite răspunsurile;

-serverul poate citi un mesaj, trimite răspunsul, sau altă combinație de read-write.

Ordinea mesajelor din fiecare stream se păstrează.

```
rpc BidiHello(stream HelloRequest) returns (stream HelloResponse) {}
```

gRPC - Definirea unui serviciu

- Pornind de la definiția unui serviciu dintr-un fișier `.proto`, gRPC furnizează **plugin-uri** pentru compilatorul protoc care permit generarea codului corespunzător clienților și serverului.
- Clienții și serverul gRPC folosesc acest API pentru implementarea funcționalității dorite:
 - În aplicația server, serverul implementează metodele declarate de serviciu, și le face disponibile clienților folosind un server gRPC. Frameworkul gRPC decodifică cererile, execută metodele remote și codifică răspunsurile.
 - În aplicația client, clientul are un obiect local (stub, proxy) care implementează aceleași metode ca și serviciul.
 - Clientul poate apela aceste metode ca și cum ar fi apeluri locale, folosind parametrii corespunzători.

gRPC - Timeout

- gRPC permite clienților să specifice cât timp sunt dispuși să aștepte ca un apel la distanță să își încheie execuția. Dacă durată specificată a fost depășită, apelul la distanță se va încheia cu eroarea `DEADLINE_EXCEEDED`.
- Pe server se poate verifica dacă un anumit apel la distanță a depășit durată sau cât timp mai are la dispoziție.
- Atât clientul cât și serverul pot anula execuția unui apel la distanță în orice moment. Anularea duce la oprirea imediată a execuției apelului la distanță.
- Anularea nu este o operație de tip “undo”: modificările făcute înainte de anulare nu vor fi anulate și ele.

gRPC - Terminare, canale de comunicare

- **Terminarea unui apel** - clientul și serverul determină independent succesul execuției unui apel la distanță, iar concluziile lor pot să difere.
 - Exemplu 1: un apel RPC se poate termina cu succes pe server (“I have sent all my responses!”), dar poate eșua pe client (“The responses arrived after my deadline!”).
 - Exemplu 2: serverul poate decide terminarea înainte ca clientul să fi terminat de trimis toate cererile.
- **Canal de comunicare gRPC** - furnizează o conexiune la un server gRPC specificând adresa și portul și se folosește la instanțierea unui proxy (client stub). Clienții pot specifica parametrii pentru a modifica comportamentul implicit (setarea opțiunii de compresie a mesajelor, etc).
- **Un canal are asociată o stare** (dacă este conectat, dacă este idle, etc.)

gRPC - Exemplu

- O aplicație simplă client/server:
 - Obiectul remote are o metodă care transformă un text în text cu litere mari și adaugă data și ora la care a fost primit textul.
 - Clientul obține o referință la obiectul remote, apelează metoda și tipărește rezultatul primit.

Bibliografie

- Protocol Buffers

<https://developers.google.com/protocol-buffers/>

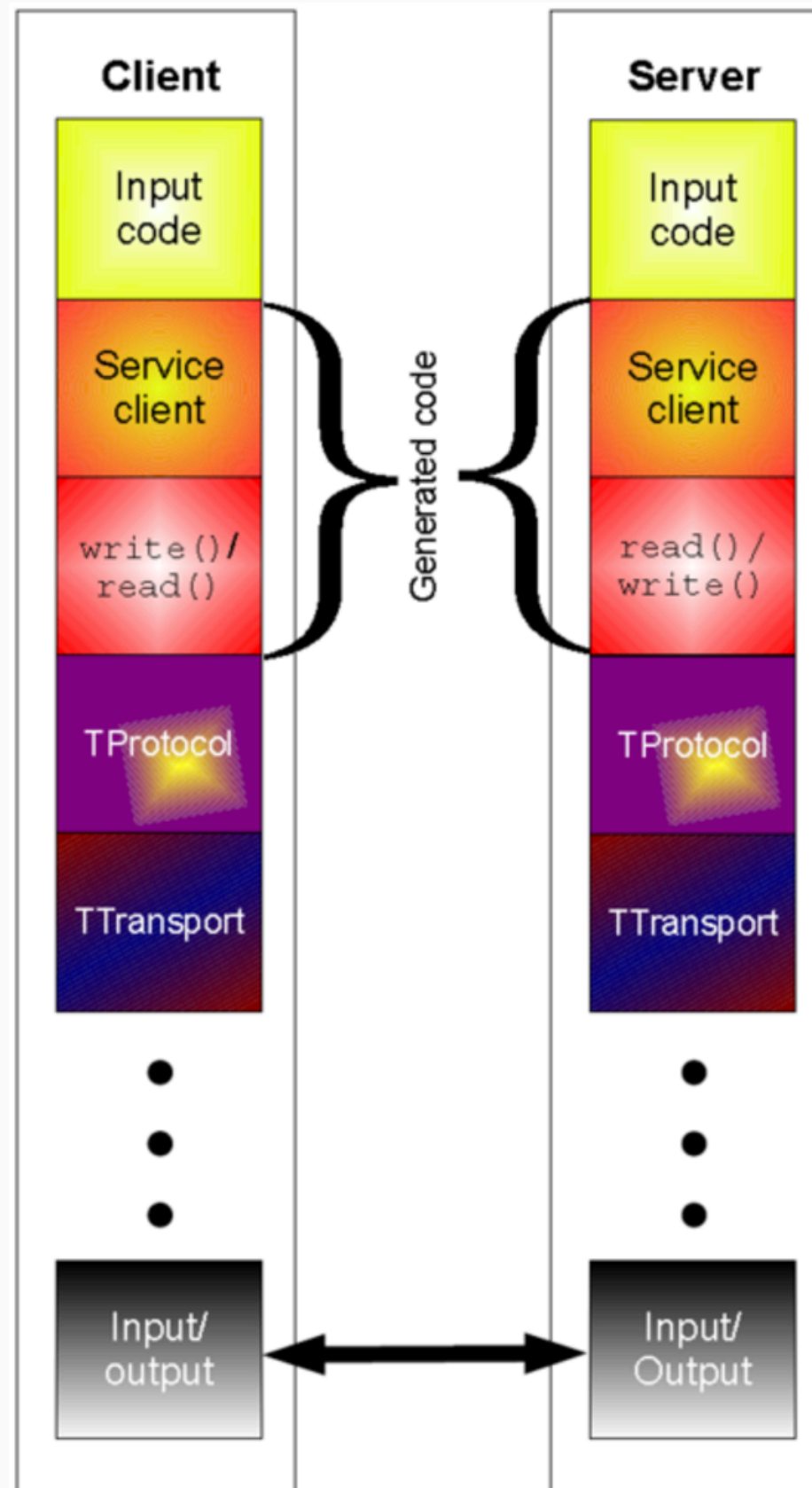
<https://github.com/google/protobuf>

- gRPC

<http://www.grpc.io/>

Apache Thrift

- **Thrift** este o tehnologie cross-platform pentru RPC.
- Thrift oferă separare clară între nivelul transport, serializarea datelor și logica aplicației.
- Thrift a fost dezvoltat inițial de Facebook, acum este un proiect open-source găzduit de Apache.
- Apache Thrift conține un set de unelte de generare de cod care permit utilizatorilor să dezvolte clienți și servere RPC doar prin definirea tipurilor de date necesare și a interfețelor serviciilor într-un fișier IDL **.thrift**.
- Folosind acest fișier ca și date de intrare, se generează automat cod sursă pentru clienți și servere RPC în diferite limbaje de programare care permit aplicațiilor să comunice indiferent de limbajul folosit pentru scrierea clientului/serverului.
- Thrift suportă diferite limbaje de programare: C++, Java, C#, Python, PHP, Ruby.



Thrift Architecture.

Image from wikipedia.org

Apache Thrift

- Componente cheie:

- tipuri de date
- transport
- protocol
- versioning
- processors

- Tipuri de date Thrift

- Sistemul de tipuri de date Thrift nu introduce tipuri speciale dinamice sau obiecte wrapper.
- Programatorul nu trebuie să scrie cod pentru serializarea tipurilor de date sau pentru transportul datelor.
- Conține tipuri de bază, structuri, containere.
- Tipuri de bază:
 - *bool* - valoarea booleană: true sau false
 - *byte* - a signed byte
 - *i16* - întreg cu semn pe 16-biți
 - *i32* - întreg cu semn pe 32-biți
 - *i64* - întreg cu semn pe 64-biți
 - *double* - real pe 64-biți
 - *string*
 - *binary* - șir de octeți folosit pentru reprezentarea blob-urilor.

Tipuri Thrift -Structuri

- O structură Thrift definește un obiect comun tuturor limbajelor ce poate fi transmis prin rețea.
- O structură este echivalentul unei clase dintr-un limbaj orientat obiect.
- Structura conține o mulțime de câmpuri care au un tip definit și un identificator unic asociat.
- Sintaxa asemănătoare structurii C.
- Câmpurile pot fi adnotate cu un identificator numeric unic (valoare întreagă) și o valoare implicită (opțional). Identificatorii trebuie să fie unici în cadrul structurii.
- Dacă identificatorii câmpurilor sunt omiși, li se vor atribui automat valori.

```
struct Example {  
    1:i32 number=10,  
    2:i64 bigNumber,  
    3:double decimals,  
    4:string name="thrifty"  
}
```

Tipuri Thrift -Containere

- Containerele Thrift sunt containere cu tip care se vor mapa la cele mai folosite containere din limbajele de programare corespunzătoare.
- Sunt parametrizate folosind stilul Java Generics/C++ template.
- Trei tipuri de containere disponibile:
 - **list<type>** : o listă de elemente.
 - Se mapează la STL **vector**, Java **ArrayList**, sau orice alt tablou din limbajele scripting. Poate conține duplicate.
 - **set<type>** : O mulțime neordonată de elemente.
 - Se mapează la STL **set**, Java **HashSet**, **set** în Python, sau dicționare native în PHP/Ruby.
 - **map<type1 , type2>** :Un dicționar cu chei unice
 - Se mapează la STL **map**, Java **HashMap**, PHP associative array, sau Python/Ruby dictionary.
- Elementele din container pot fi de orice tip valid Thrift, inclusiv alte containere sau structuri.
- În codul generat corespunzător limbajului de programare dorit, fiecare definiție va conține două metode, **read** și **write**, folosite pentru serializarea și transportul obiectelor folosind Thrift TProtocol.

Thrift - Excepții

- Excepțiile Thrift sunt sintactic și funcțional echivalente cu structurile Thrift doar că sunt declarate folosind `exception` în loc de `struct`.
- Clasele generate vor moșteni din clasele de bază corespunzătoare excepțiilor în limbajul de programare folosit, pentru a se putea folosi în mod transparent cu mecanismul de tratare a excepțiilor din limbajul respectiv.

Servicii Thrift

- Serviciile Thrift se definesc folosind tipurile Thrift.
- Definirea unui serviciu este echivalentă semantic cu definirea unei interfețe într-un limbaj de programare orientat pe obiecte.
- Compilatorul Thrift va genera stub-uri client și server complet funcționale care implementează interfața.
- Lista parametrilor și lista excepțiilor sunt implementate ca și structuri Thrift.

```
service <name> {  
    <returntype> <name>(<arguments>) [throws (<exceptions>)]  
    ...  
}
```

```
service StringCache {  
    void set(1:i32 key, 2:string value),  
    string get(1:i32 key) throws (1:KeyNotFound knf),  
    void delete(1:i32 key)  
}
```

Nivelul transport Thrift

- Nivelul transport este folosit de codul generat automat pentru a ușura transmiterea datelor între clienți și server.
- Thrift se folosește de obicei peste TCP/IP ca și nivel de bază pentru comunicare.
- Codul generat Thrift trebuie să știe doar cum să scrie și cum să citească datele.
- Originea sau destinația datelor sunt irelevante: poate fi socket, memorie partajată sau un fișier pe discul local.
- Interfața Thrift **TTransport** conține metodele:
 - **open** deschiderea transportului
 - **close** închiderea transportului
 - **isOpen** verifică dacă transportul este deschis
 - **read** citește date
 - **write** scrie date
 - **flush** forțează scrierea datelor păstrate în zona tampon.

Nivelul transport Thrift

- Interfață **TServerTransport** folosită pentru crearea și acceptarea obiectelor transport:
 - **open** deschidere
 - **listen** așteaptă conexiuni de la clienți
 - **accept** returnează un nou obiect transport (când s-a conectat un client nou)
 - **close** închidere
- Interfața transport este proiectată pentru implementare ușoară în orice limbaj de programare.
- Se pot defini noi mecanisme de transport:
 - Clasa **TSocket** este implementată în toate limbajele de programare suportate. Oferă o modalitate simplă de comunicare cu un socket TCP/IP.
 - Clasa **TFileTransport** este o abstractizare a unui stream ce reprezintă un fișier de pe discul local. Poate fi folosită pentru a salva cererile clienților într-un fișier de pe disc.

Thrift - Protocol

- Thrift cere respectarea unei anumite structuri a mesajelor când sunt transmise prin rețea, dar **nu știe de protocolul efectiv folosit pentru serializarea/codificarea acestora.**
- Nu contează dacă datele sunt serializate folosind XML, human-readable ASCII (stringuri) sau octeți, dacă mecanismul suportă o mulțime de operații prestabilite care permit citirea și scrierea datelor.
- Interfața Thrift Protocol suportă:
 - *trimiterea mesajelor bidirectional,*
 - *codificarea tipurilor de bază, a structurilor și a containerelor.*
- **Are o implementare care folosește un protocol binar.**
- **Scrie toate datele într-un format binar:**
 - **Tipurile întregi** sunt **convertite într-un format rețea independent de limbaj.**
 - **Stringurile** au **adăugate la început lungimea lor în număr de octeți.**
 - **Mesajele și antetul câmpurilor** sunt **scrise folosind construcții de serializare a datelor întregi.**
 - **Numele câmpurilor nu sunt serializate,** **identificatorii asociați** sunt **suficienți pentru reconstruirea datelor.**

Thrift - Protocol

```
writeMessageBegin(name, type, seq)
writeMessageEnd()
writeStructBegin(name)
writeStructEnd()
writeFieldBegin(name, type, id)
writeFieldEnd()
writeMapBegin(ktype, vtype, size)
writeMapEnd()
...
name, type, seq = readMessageBegin();
readMessageEnd()
name = readStructBegin()
readStructEnd()
name, type, id = readFieldBegin()
readFieldEnd()
k, v, size = readMapBegin()
readMapEnd()
...
```

Thrift - Versioning

- Thrift este **robust** la schimbări de versiuni și de definiție a datelor.
- **Versioning** este implementat folosind identificatorii asociați câmpurilor:
 - Antetul unui câmp dintr-o structură Thrift este codificat folosind identificatorul unic.
 - Combinația (identificator câmp, tip câmp) este folosită pentru a identifica unic un câmp din structură.
- Limbajul IDL Thrift permite atribuirea automată de identificatori pentru câmpuri, dar se recomandă definirea explicită a acestora.
- Dacă la parsare/decodificare/deserializare se întâlnește un câmp necunoscut acesta este **ignorat** și **distrus**.
- Dacă un câmp care ar trebui să existe nu apare, programatorul poate fi notificat de lipsa acestuia (folosind structura **isset** definită în interiorul fiecărei obiect).
- Obiectul **isset** din interiorul unei structuri Thrift poate fi interogat pentru a determina existența unui anumit câmp. De fiecare dată când se primește o instanță a unei structuri, programatorul ar trebui să verifice existența unui câmp înainte de folosirea lui.

Thrift - Implementare RPC

- Instanțe a clasei `TProcessor` sunt folosite pentru a trata cererile RPC:

```
interface TProcessor {  
    bool process(TProtocol in, TProtocol out) throws TException  
}
```

- Pentru fiecare serviciu dintr-un fișier `.thrift` se generează următoarele:

- o interfață `ServiceIf` corespunzătoare serviciului
- clasa `ServiceClient` implementează `ServiceIf`

`TProtocol in`
`TProtocol out`

- clasa `ServiceProcessor` : `TProcessor`
`ServiceIf handler`
- clasa `ServiceHandler` implementează `ServiceIf`
- `TServer(TProcessor processor,`
`TServerTransport transport,`
`TTransportFactory tfactory,`
`TProtocolFactory pfactory)`

`serve()`

Thrift - Implementare RPC

- Serverul încapsulează logica corespunzătoare conexiunii, threadurilor, etc, iar obiectul de tip TProcessor se ocupă de apelul metodelor la distanță.
- Programatorul trebuie să scrie doar codul din fișierul .thrift și implementarea corespunzătoare serviciilor (ServiceHandler)
- Există mai multe implementări posibile pentru TServer:
 - TSimpleServer: server secvențial
 - TThreadedServer: server concurent (se creează câte un thread pentru fiecare conexiune)
 - TThreadPoolServer: server concurent care folosește un container de threaduri.
-

Bibliografie

- Apache Thrift

<http://thrift-tutorial.readthedocs.io/en/latest/index.html>

<https://thrift.apache.org/>

- Exem plu: Text transformer

Object/Relational Mapping (ORM)

- *Object-relational mapping* (ORM, O/RM sau O/R mapping) este o tehnică de programare pentru convertirea informațiilor/tipurilor dintr-un sistem orientat-obiect într-o bază de date relațională.
- Principiul mapării obiect-relație/înregistrare este de a delega altor instrumente managementul persistenței și de a lucra doar cu entitățile din domeniu, nu cu structurile dintr-o bază de date relațională.
- Instrumentele de mapare obiect-relație stabilesc o legătură bidirecțională între o bază de date relațională și obiectele din sistem, pe baza unei configurații și execută interogări SQL la baza de date (interogări construite dinamic).

Terminologie

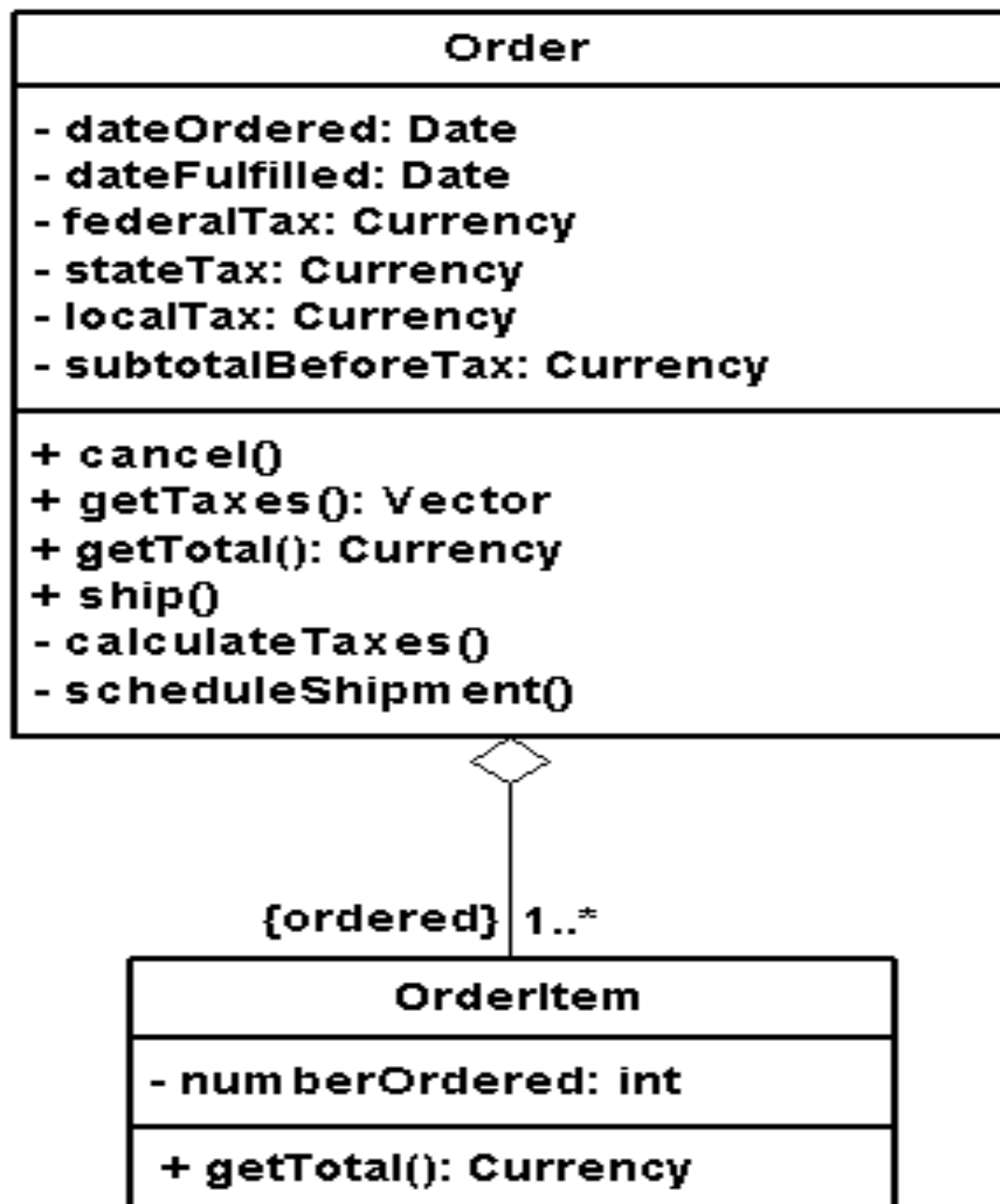
- *Mapare*. Determinarea modului în care obiectele și relațiile dintre ele vor fi păstrate într-un mediu de stocare permanent (ex. bază de date relațională, fișiere XML).
- *Proprietate*. O proprietate care poate avea asociat un atribut `firstName:string` sau o metodă prin care se determină valoarea `getTotal()`.
- *Maparea proprietății*. O mapare care descrie cum va fi stocată valoarea proprietății.
- *Maparea relațiilor*. O mapare care descrie cum vor fi persistate relațiile dintre unul sau mai multe obiecte (asociere, agregare, moștenire).

Mapări simple

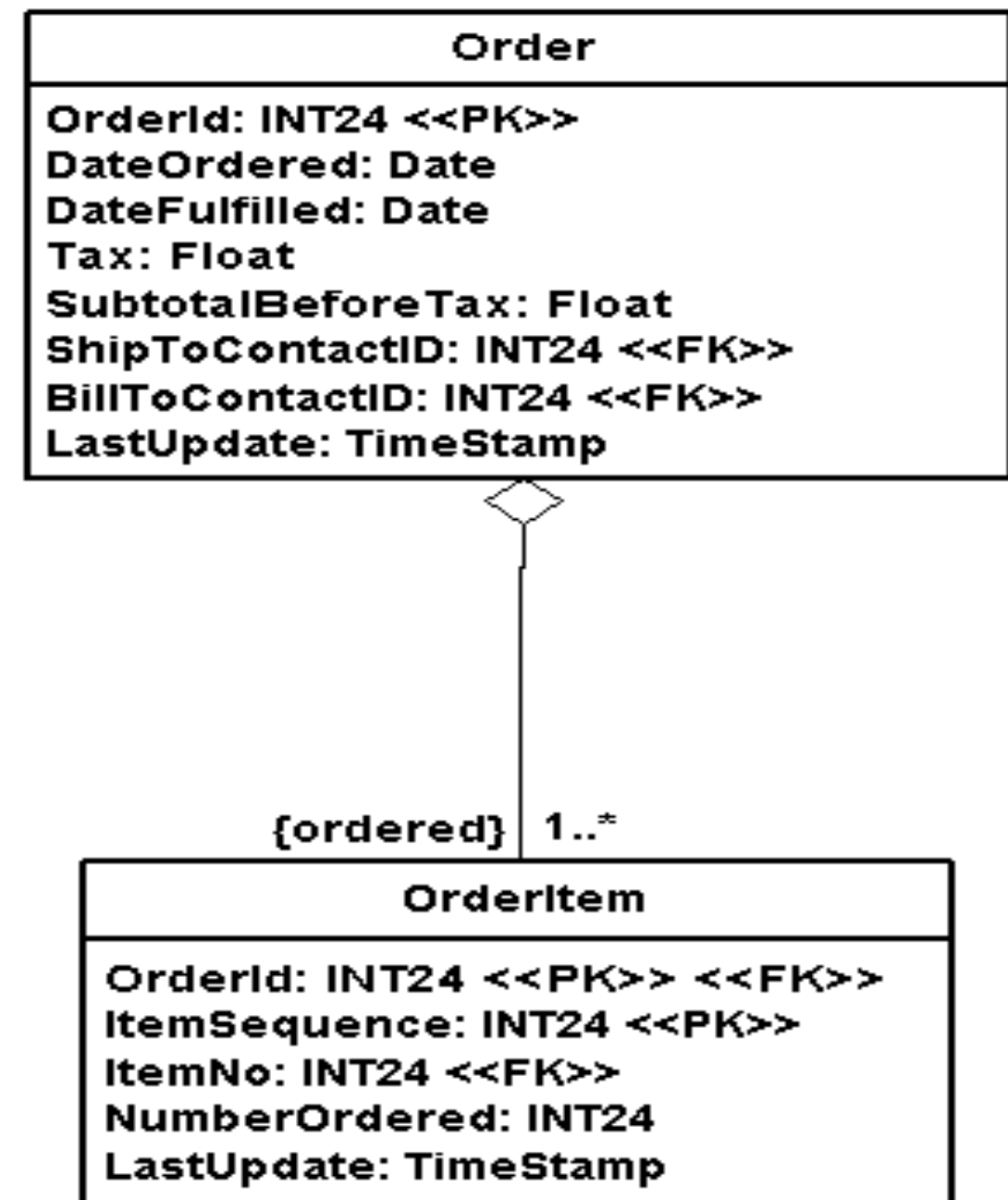
- O clasă este mapată într-o tabelă.
- Exceptând modele foarte simple, maparea unu-la-unu este rară.
- Cea mai simplă mapare este maparea unei proprietăți asociate unui singur atribut la o singură coloană din tabela corespunzătoare.
- Este și mai simplu dacă ambele (atributul și coloana) au aceleași tipuri de bază.
 - ambele sunt stringuri/date,
 - atributul este un string, coloana e de tip char(varchar),
 - atributul este un număr, iar coloana este float.

Mapări simple

<<Class Model>>



<<Physical Data Model>>



Diferențe

- Sunt mai multe attribute pentru **tax** în modelul orientat obiect - doar o singură coloană în schema relațională. Cele trei attribute din clasa **Order** ar trebui însumate și rezultatul păstrat în tabelă.
- În schema relațională apar chei, în modelul orientat obiect nu există chei. Înregistrările din tabela relațională sunt identificate în mod unic de cheia primară, iar relațiile dintre tabele sunt păstrate folosind chei străine.
- Relațiile dintre obiecte sunt păstrate prin referințe, nu prin chei străine. Pentru a putea persista obiectele și relațiilor dintre ele, obiectele trebuie să știe de valoarea cheilor păstrate în baza de date pentru a le putea identifica. Informația adițională este numită “*shadow information*”.
- Sunt folosite diferite tipuri în modelul orientat obiect și în schema relațională:
 - atributul **subTotalBeforeTax** din clasa **Order** este de tip **Currency**
 - coloana **SubTotalBeforeTax** din tabela **Order** este de tip float.
 - Pentru a implementa maparea trebuie să putem converti între cele două reprezentări fără a pierde informații.

Shadow Information

- *Shadow information* sunt orice informații pe care obiectele trebuie să le păstreze (pe lângă informațiile normale) pentru a putea fi persistate.
- Include:
 - *Cheia primară*: în special când cheia primară este o cheie surogat care nu are altă semnificație în domeniu.
 - *Informații pentru controlul concurenței*: timestamps sau incremental counters.
 - *Informații pentru păstrarea versiunii*: *versioning* numbers.
- Exemplu: tabela **Order** are coloana **OrderID** folosită ca și cheie primară și coloana **LastUpdate** folosită pentru controlul concurenței care nu apar în clasa **Order**.

Mapare Metadata

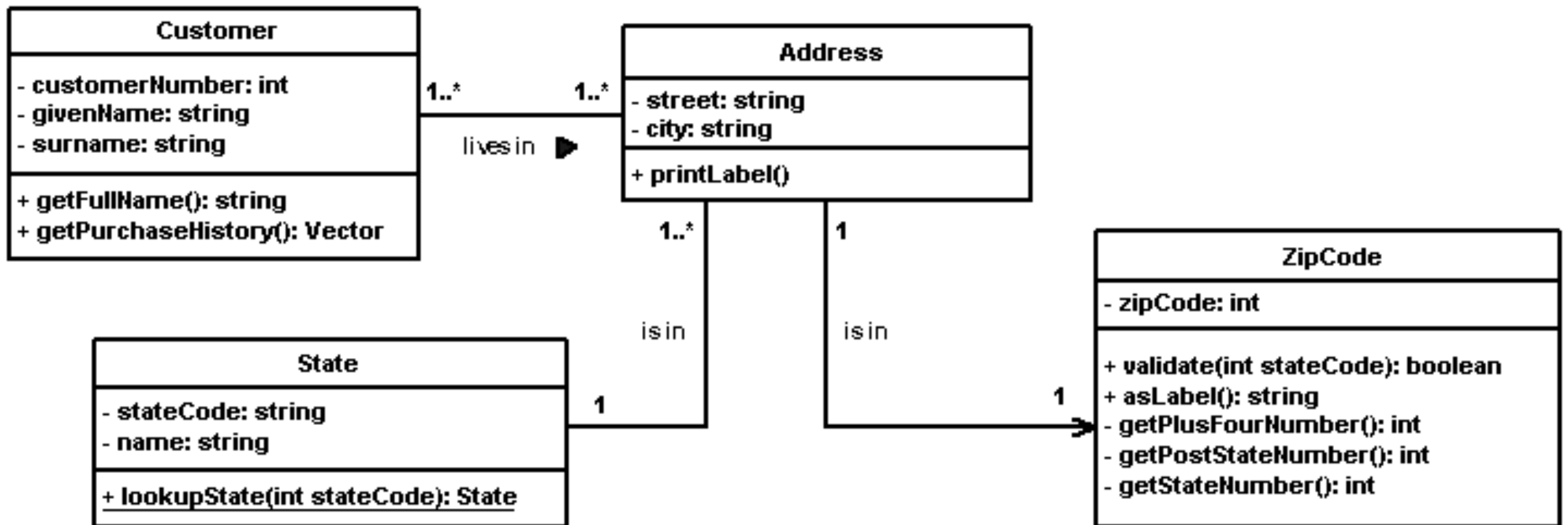
- Metadata păstrează informații despre date. Maparea metadatelor descrie modul în care metadatele reprezentând proprietățile sunt mapate la metadatele corespunzând tabelelor.

Proprietate (OO)	Coloana (BD)
Order.orderID	Order.OrderID
Order.dateOrdered	Order.DateOrdered
Order.dateFulfilled	Order.DateFulfilled
Order.getTotalTax()	Order.Tax
Order.subtotalBeforeTax	Order.SubtotalBeforeTax
Order.shipTo.personID	Order.ShipToContactID
Order.billTo.personID	Order.BillToContactID
Order.lastUpdate	Order.LastUpdate
OrderItem.ordered	OrderItem.OrderID
Order.orderItems.position(orderItem)	OrderItem.ItemSequence
OrderItem.item.number	OrderItem.ItemNo
OrderItem.numberOrdered	OrderItem.NumberOrdered

ORM Impedance Mismatch

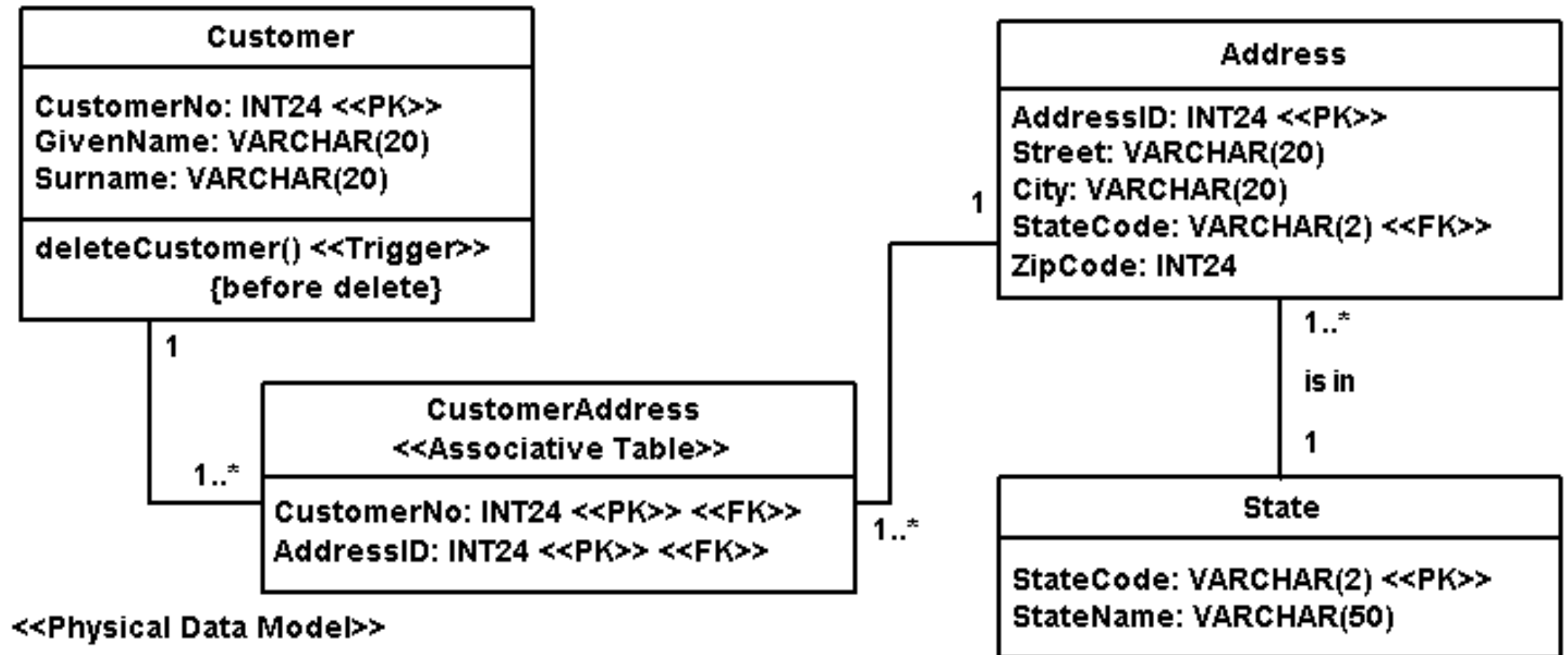
- Paradigma orientată obiect promovează dezvoltarea aplicațiilor folosind obiecte care păstrează date, dar conțin și logica aplicației.
- Bazele de date relaționale stochează datele în tabele și manipulează datele folosind proceduri stocate și interogări SQL.
- Diferențele dintre cele două abordări au fost numite: *object-relational impedance mismatch* sau doar *impedance mismatch*.
- Ex. în paradigma orientată obiect obiectele sunt traversate folosind relațiile dintre ele, în paradigma relațională se folosește operația de join.
- Tipurile de date diferite în limbajele orientate obiect și bazele de date relaționale:
 - Java: string și int - Oracle: varchar și smallint.
 - Java: colecții - Oracle: tabele
 - Java: obiecte - Oracle: blobs

ORM Impedance Mismatch



Copyright 2002-2006 Scott W. Ambler

ORM Impedance Mismatch



Strategii pentru Impedance Mismatch

- Maparea moștenirii
- Maparea relațiilor dintre obiecte
- Maparea proprietăților statice

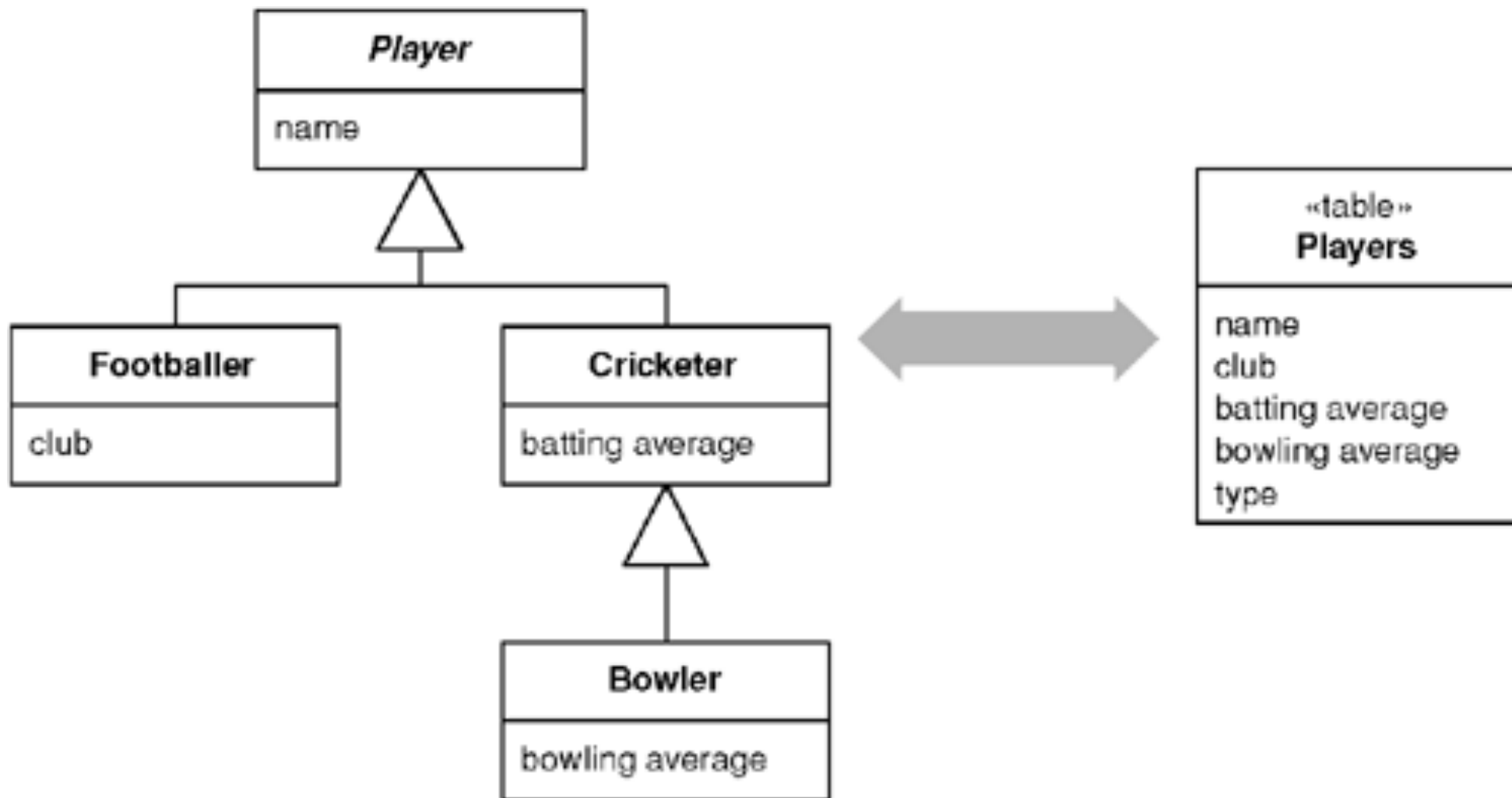
Maparea moștenirii

- Bazele de date relaționale nu suportă moștenirea.
- Programatorul trebuie să mapeze moștenirea dintre entitățile din modelul orientat obiect într-o bază de date relațională.
- Tehnici:
 - Maparea ierarhiei de clase într-o singură tabelă.
 - Maparea fiecărei clase concrete în tabela ei.
 - Maparea fiecărei clase într-o tabelă.
 - Maparea claselor într-o structura de tabele generică.

Moștenirea - O singură tabelă

- Reprezentarea unei ierarhii de clase (moștenire) **ca și o singură tabelă** cu coloane pentru toate atributele din toate clasele din ierarhie.
- Fiecare clasă păstrează informațiile relevante pentru ea într-o înregistrare din tabelă. Coloanele care nu sunt relevante rămân goale.
- Când se încarcă un obiect din tabelă, instrumentul ORM trebuie să știe ce clasă să instanțieze.
- În tabelă se adaugă o coloană care indică ce clasă ar trebui instanțiată (numele clasei sau un cod):
 - Codul trebuie interpretat în codul sursă pentru a putea face maparea cu clasa corespunzătoare.
 - Numele clasei poate fi folosit direct pentru instanțiere (folosind reflecție).

Moștenirea - O singură tabelă



Moștenirea - O singură tabelă

```
Footballer fb=new Footballer("A A", "ABC")
Cricketer cr=new Cricketer("C C", 23);
Bowler bw=new Bowler("B B", 21, 47);
Footballer fb2=new Footballer("D D", "BGD");
Bowler bw2=new Bowler("H H", 12, 23);
```

Players

PK	Name	Club	BattlingAvg	BowlingAvg	Type
1	A A	ABC			Footballer
2	C C		23		Cricketer
3	B B		21	47	Bowler
4	D D	BGD			Footballer
5	H H		12	23	Bowler

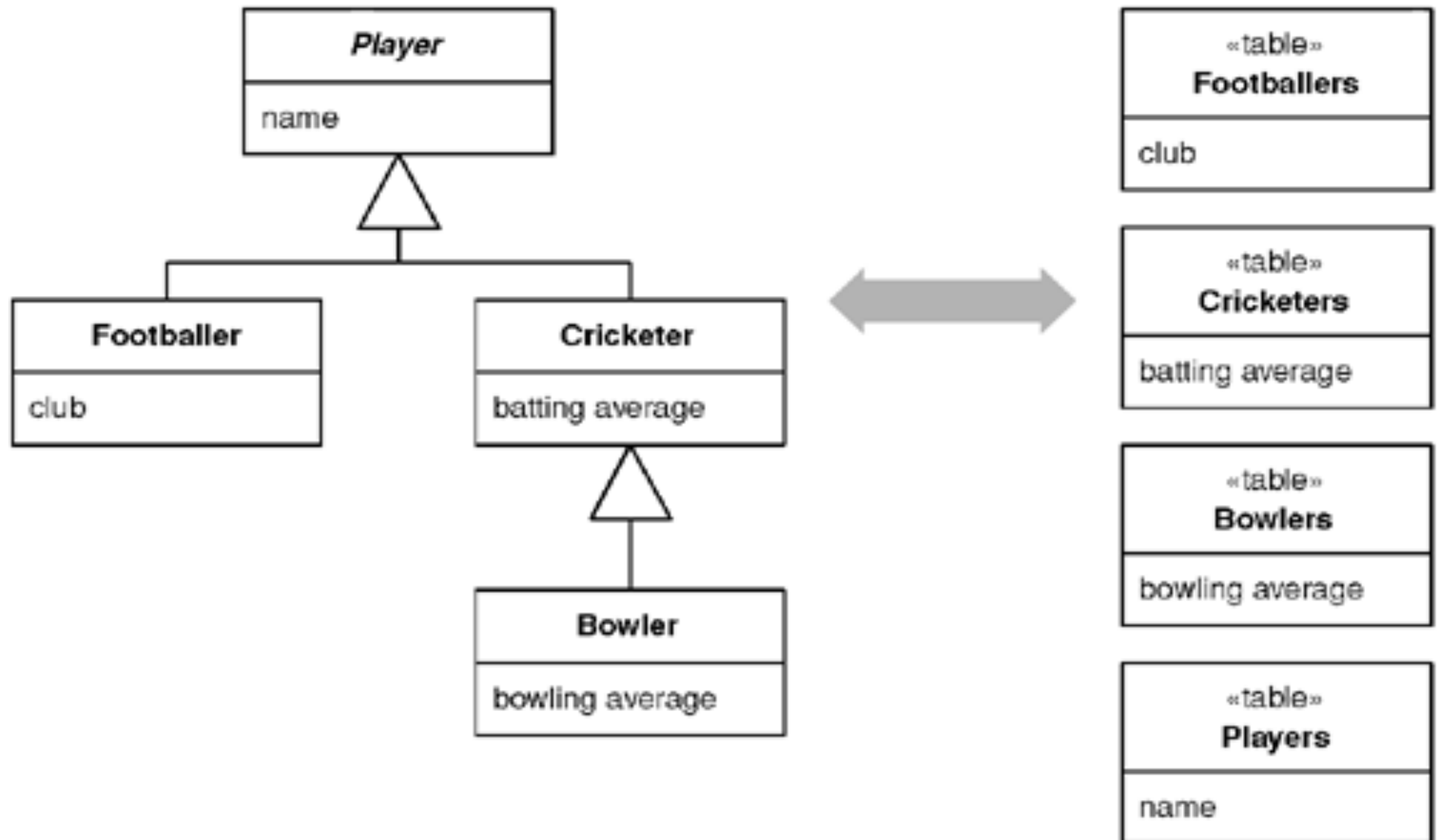
Moștenirea - O singură tabelă

- **Avantaje:**
 - Există doar o singură tabelă în baza de date.
 - Nu e nevoie de operații join pentru regăsirea informației.
 - Orice refactorizare care mută attributele în ierarhie nu necesită modificarea bazei de date.
- **Dezavantaje:**
 - Nu toate câmpurile din tabelă sunt relevante (depinde de tipul clasei). Este confuz pentru cei care folosesc tabelele direct (fără instrumentul ORM).
 - Coloanele folosite doar de subclase duc la spațiu nefolosit (multe coloane goale).
 - Tabela poate deveni prea mare, cu mulți indecși și blocări dese ale tablei. Poate afecta performanța.
 - Există un singur spațiu de nume pentru câmpuri, dezvoltatorul trebuie să se asigure că se vor folosi nume diferite în tabelă.
 - Adăugarea numelui clasei (prefix, postfix) poate ajuta. (ex. NumeClasă_NumeProprietate)

Tabelă pentru fiecare clasă

- Fiecare clasă din ierarhie are tabela ei.
- Atributele din clasă se mapează direct la coloanele corespunzătoare din tabelă.
- *Problemă:* Cum se leagă înregistrările din tabele?
 - *Soluția A:* folosirea cheii primare atât în tabela corespunzătoare clasei de bază cât și în clasa derivată. Deoarece clasa de bază are câte o înregistrare pentru fiecare înregistrare din clasele derivate, cheia primară va fi unica între toate tabele.
 - *Soluția B.* Fiecare tabelă să aibă cheia primară proprie, și folosirea cheii străine pentru a păstra legătura cu tabela corespunzătoare clasei de bază.
- *Provocare:* încărcarea/regăsirea informațiilor din mai multe tabele în mod eficient.
 - Operații de join între diferite tabele
 - Operațiile de join între mai mult de 3 sau 4 tabele sunt lente din cauza modului în care bazele de date optimizează operațiile interne.
- Interogările asupra bazei de date sunt dificile.

Tabelă pentru fiecare clasă



Tabelă pentru fiecare clasă

```
Footballer fb=new Footballer("A A", "ABC")
Cricketer cr=new Cricketer("C C", 23);
Bowler bw=new Bowler("B B",21, 47);
Footballer fb2=new Footballer("D D", "BGD");
Bowler bw2=new Bowler("H H",12, 23);
```

Players

PK	Name
1	A A
2	C C
3	B B
4	D D
5	H H

Footballers

PK	Club
1	ABC
4	BGD

Cricketers

PK	BattlingAvg
2	23
3	21
5	12

Bowlers

PK	BowlingAvg
3	47
5	23

Tabelă pentru fiecare clasă

- Avantaje:

- Toate coloanele sunt relevante pentru fiecare înregistrare, tabelele sunt mai ușor de înțeles și nu se folosește spațiu în mod ineficient.
- Relația dintre entitățile din modelul orientat obiect și baza de date relațională este ușor de înțeles.

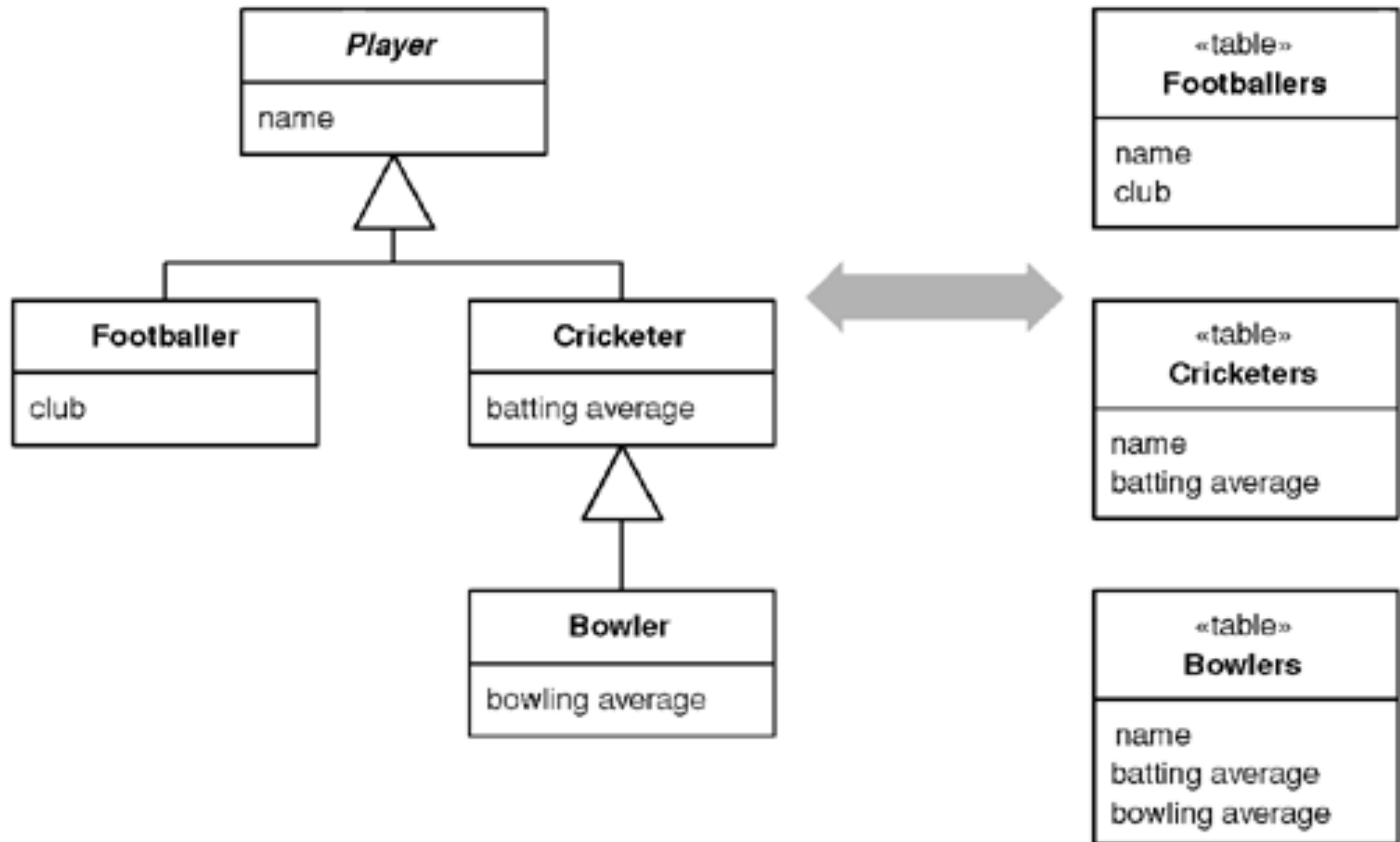
- Dezavantaje:

- Este necesară folosirea mai multor tabele pentru a încărca un obiect din mediu **persistent** (operație join sau mai multe interogări și folosirea memoriei).
- Orice refactorizare (mutarea câmpurilor în ierarhia de clase) cauzează modificarea structurii bazei de date.
- Tabelele corespunzătoare claselor de bază pot cauza probleme de performanță din cauza accesării dese.
- Normalizarea poate duce la înțelegerea dificilă a interogărilor ad-hoc.

Tabelă pentru fiecare clasă concretă

- Fiecare clasă concretă (non-abstract) din ierarhie are tabela ei.
- Fiecare tabelă conține coloane pentru toate proprietățile din ierarhie până la ea. ***Atributele din clasa de bază sunt duplicate în tabelele corespunzătoare subclaselor.***
- Este responsabilitatea programatorului de a se asigura că cheile sunt unice nu doar în tabela corespunzătoare clasei dar și între toate tabelele asociate ierarhiei.

Tabelă pentru fiecare clasă concretă



Tabelă pentru fiecare clasă concretă

```
Footballer fb=new Footballer("A A", "ABC")
Cricketer cr=new Cricketer("C C", 23);
Bowler bw=new Bowler("B B",21, 47);
Footballer fb2=new Footballer("D D", "BGD");
Bowler bw2=new Bowler("H H",12, 23);
```

Footballers

PK	Name	Club
1	A A	ABC
4	D D	BGD

Cricketers

PK	Name	BattlingAvg
2	C C	23

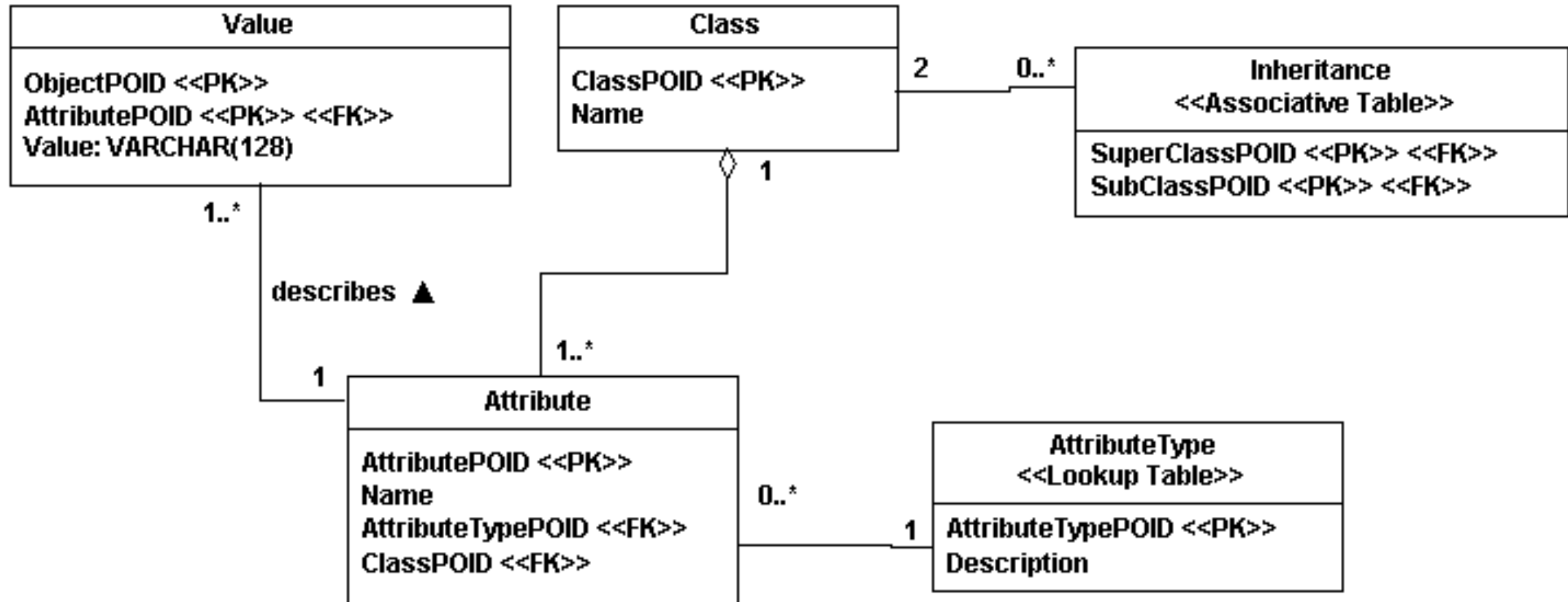
Bowlers

PK	Name	BattlingAvg	BowlingAvg
3	B B	21	47
5	H H	12	23

Tabelă pentru fiecare clasă concretă

- **Avantaje:**
 - Fiecare tabelă păstrează toate informațiile relevante și nu are câmpuri irelevante. Este ușor de înțeles și de alte aplicații care nu folosesc obiecte.
 - Nu este nevoie de operații join pentru citirea datelor.
 - Fiecare tabelă este accesată doar când clasa respectivă este accesată. Performanța este mai bună.
- **Dezavantaje:**
 - Gestiunea dificilă a cheilor primare.
 - Nu pot fi constrânse relațiile către clasele abstracte.
 - Dacă câmpurile din modelul obiectual sunt mutate în ierarhie, trebuie modificate definițiile tabelelor.
 - Dacă se modifică un câmp dintr-o clasă de bază, trebuie modificate toate tabelele corespunzătoare subclaselor, pentru că aceste câmpuri sunt duplicate.
 - O operație de căutare folosind clasa de bază, necesită căutări în toate **tabelele** (accesări multiple ale bazei de date sau o operație de join complicată).

Tabelle generiche



Tabele generice

- Avantaje:
 - Poate fi extinsă pentru a oferi suport pentru o gamă largă de mapări, inclusiv maparea relațiilor.
 - Este flexibilă, permite modificarea ușoară a modului în care sunt păstrate obiectele (trebuie modificate doar metadatele din tabelele *Class*, *Inheritance*, *Attribute* și *AttributeType*).
- Dezavantaje:
 - Este fezabilă doar pentru date de dimensiuni mici, deoarece necesită accesări dese ale bazei de date doar pentru reconstruirea unui singur obiect).
 - Interogările pot fi dificile deoarece necesita accesarea mai multor înregistrări pentru un singur obiect.

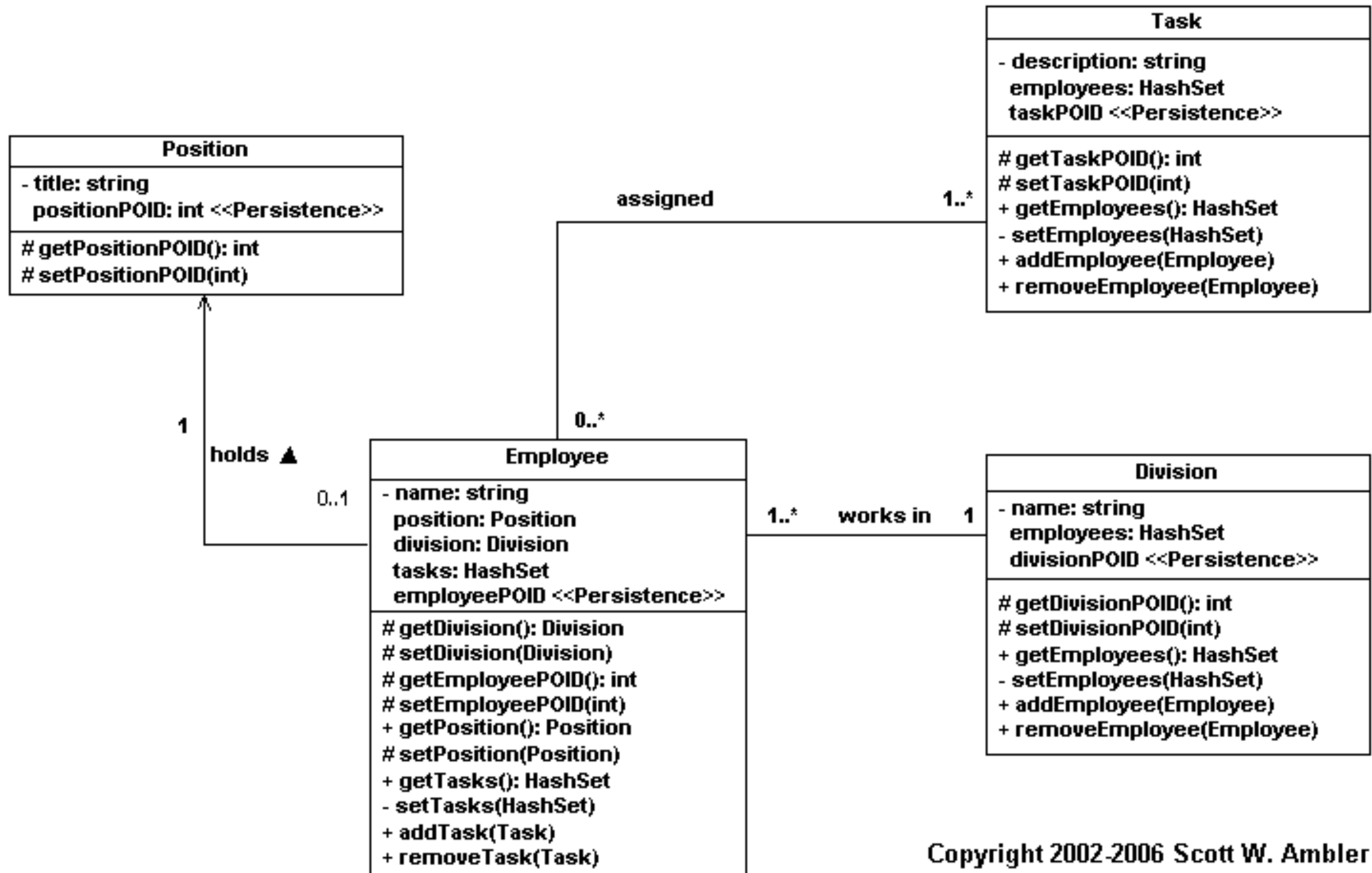
Maparea relațiilor dintre obiecte

- *Problema*: modul în care paradigma orientată obiect și modelul relațional tratează legăturile dintre obiecte/înregistrari, care cauzează două situații.
 1. *Diferența reprezentării*. Obiectele tratează legăturile prin păstrarea referințelor care există în timpul execuției (**referințele sunt temporare**). Bazele de date relaționale tratează legăturile prin păstrarea cheilor în tabele (**cheile sunt permanente**).
 2. *Obiectele pot folosi colecții pentru a gestiona mai multe referințe într-un singur atribut*. Normalizarea obligă ca toate legăturile/valorile să nu fie multiple. Se inversează structura de date dintre obiecte și tabele.
- Exemplu: Un obiect *Order* are o colecție de obiecte de tip *line item* care nu păstrează o referință către obiectul de tip order.
 - ★ Structura tabelii este inversă, înregistrările *line item* includ o cheie străină către înregistrarea *order* corespunzătoare (câmpurile dintr-o înregistrare nu pot fi multivaloare).

Tipuri de relații

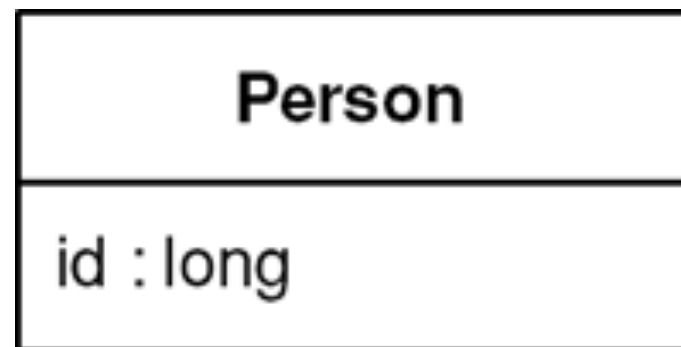
- Două categorii de relații între obiecte importante la mapare:
- **Multiplicitatea**. Include 3 tipuri:
 - Relații *unu-la-unu*. Maximul multiplicității la ambele capete este 1.
 - Relații *unu-la-n (sau n-la-unu)*. Multiplicitatea la unul dintre capete este 1, la celălalt capăt este n.
 - Relații *n-la-n*. Maximul multiplicității la ambele capete este mai mare decât 1.
- **Direcția**. Include 2 tipuri:
 - Relații *unidirecționale*. Un obiect știe de obiectul (obiectele) cu care are o legătură, dar celălalt obiect (celelalte obiecte) nu știe (nu știu) de el.
 - Relații *bidirecționale*. Ambele obiecte aflate într-o relație știu unul de celălalt.

Tipuri de relații



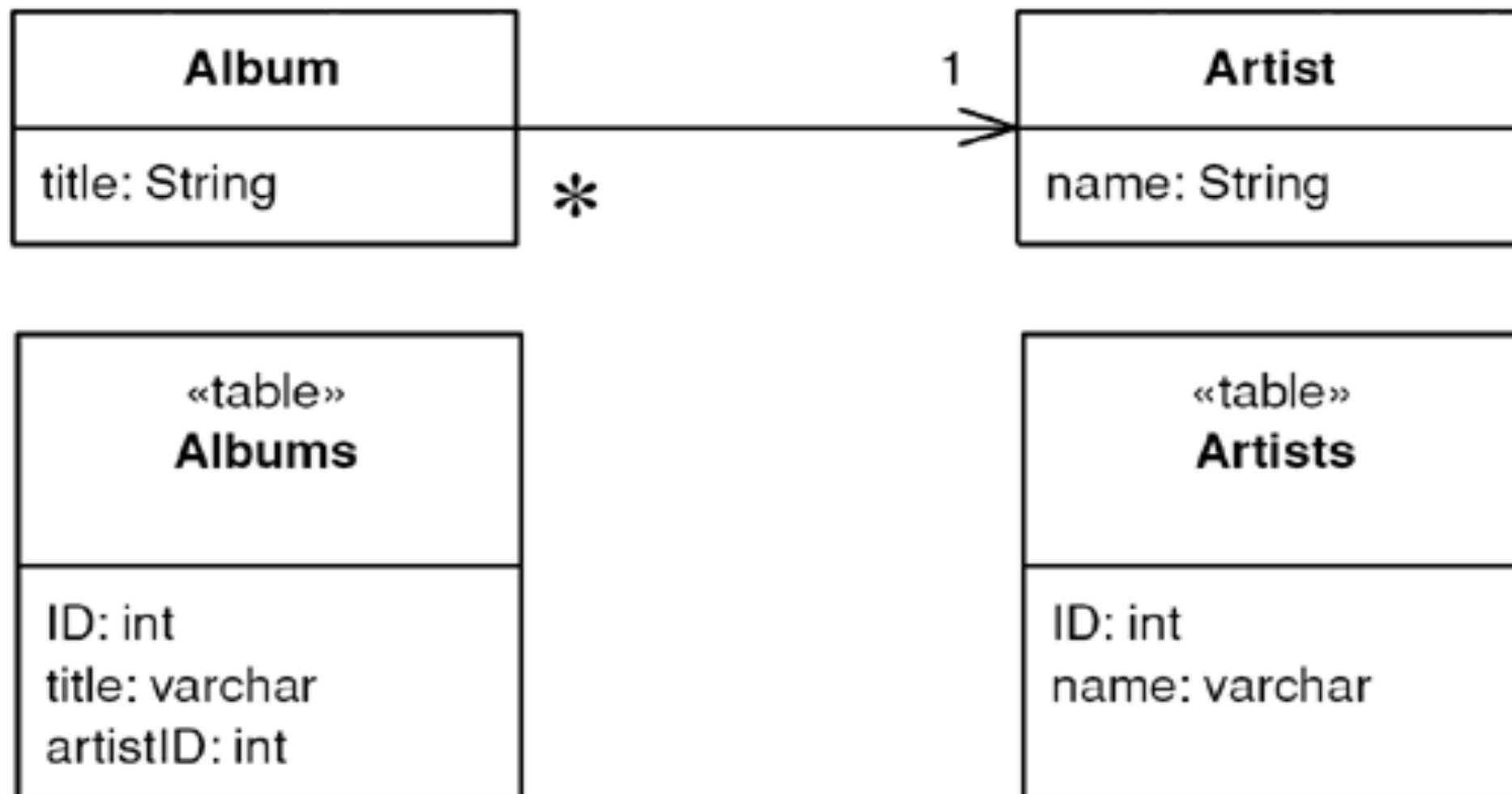
Șablonul *Identity*

- Salvează un identificator (ID) din baza de date într-un obiect pentru a păstra legătura dintre un obiect în memorie și o înregistrare din baza de date.
- Cheia primară dintr-o bază de date relațională este păstrată printre attributele obiectului.
- Șablonul ar trebui folosit când există o mapare între obiectele din memorie și înregistrările din baza de date (cheia primară este diferită de attributele din obiect).



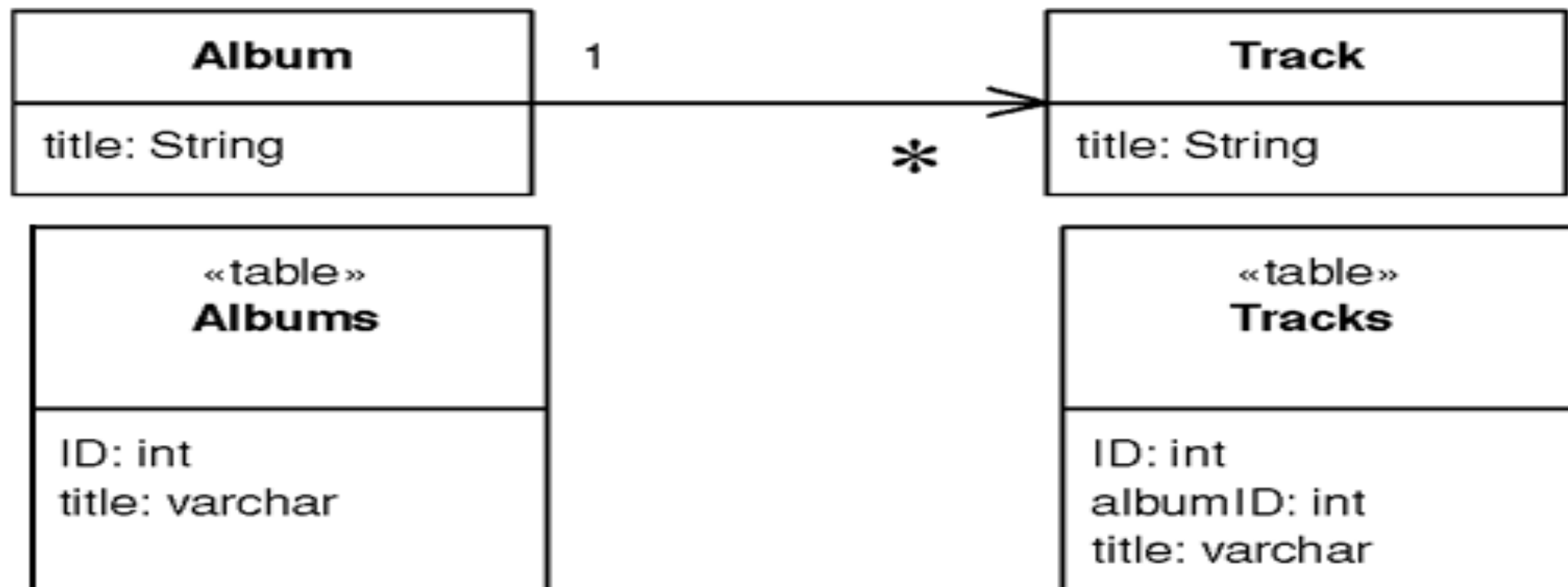
Maparea cheii străine

- Mapează o asociere dintre obiecte ca și cheie străină între tabelele dintr-o bază de date relațională.
- Fiecare obiect conține cheia din tabela corespunzătoare.
- Dacă două obiecte sunt legate printr-o relație de asociere, relația poate fi înlocuită printr-o cheie străină în baza de date.



Maparea cheii străine

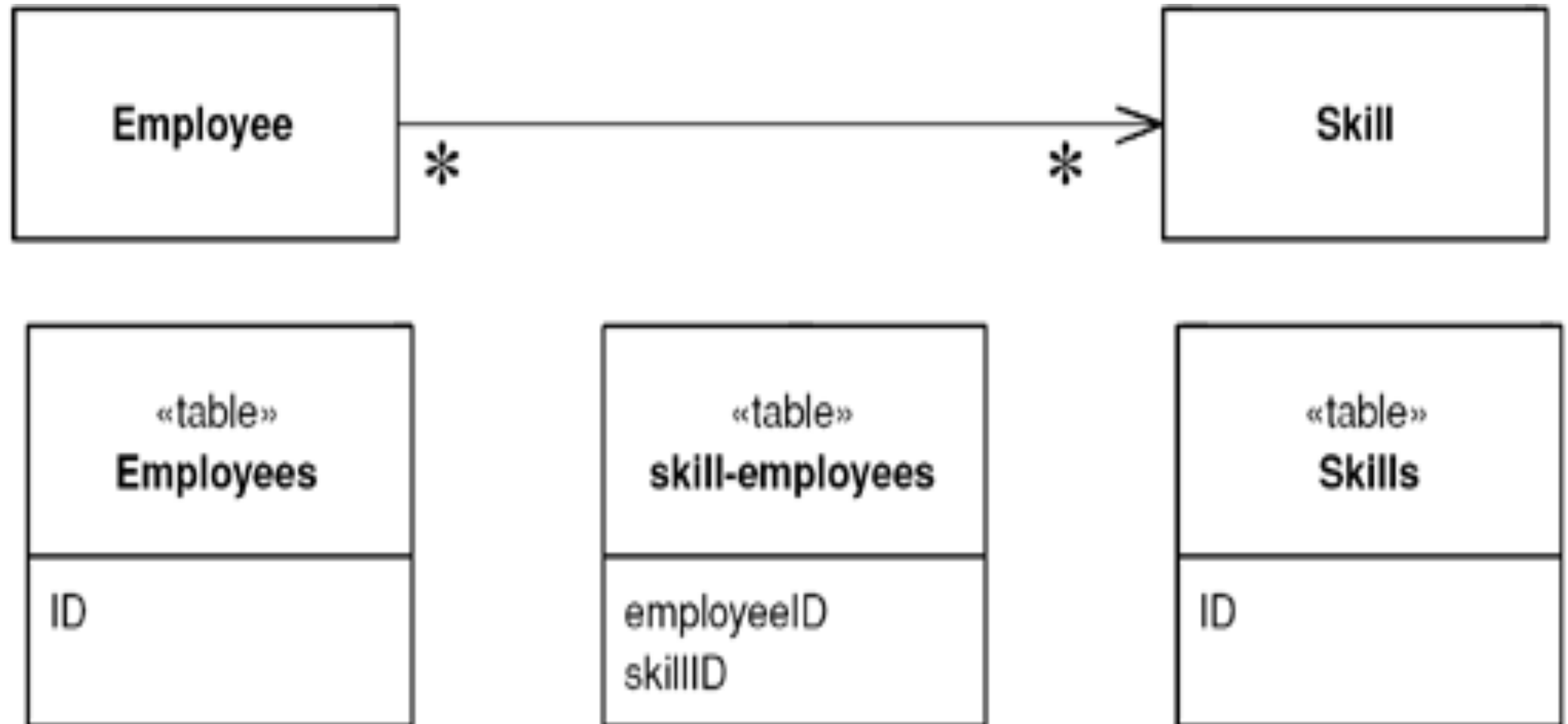
- Maparea unei colecții de obiecte.
- Într-o bază de date relațională nu se poate salva o colecție, trebuie inversată direcția referinței.
- Maparea cheii străine poate fi folosită pentru aproape toate asocierile dintre clase. Nu poate fi folosită pentru asocierea *n-la-n*.



Maparea folosind o tabelă de asociere

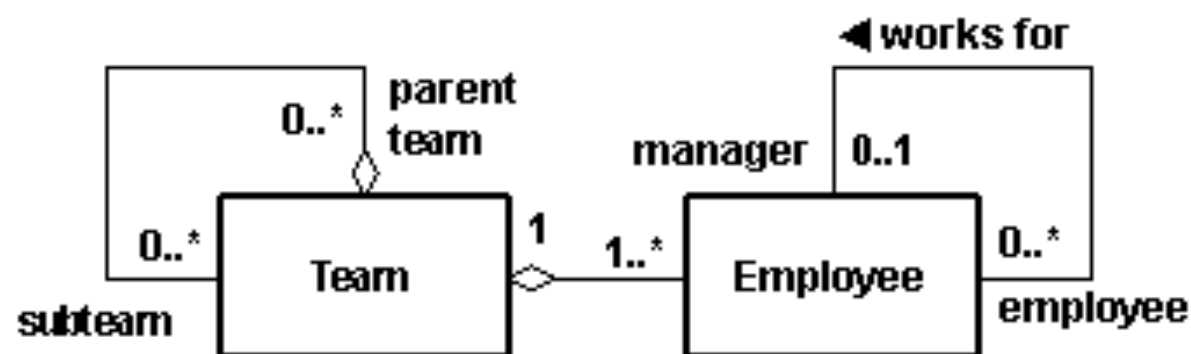
- Salvează o asociere ca o tabelă cu chei străine către tabelele legate prin asociere.
- Obiectele pot păstra mulțimi de valori folosind colecții. Bazele de date relaționale nu au această caracteristică și sunt restrictionate la câmpuri cu o singură valoare.
- Ideea este de a crea o tabelă de legătură/asociere pentru a stoca asocierea.
- Tabela are doar două coloane corespunzătoare cheilor străine, conține câte o înregistrare pentru fiecare pereche de obiecte asociate.
- Tabela de legătura nu are echivalentul unui obiect în memorie (nu are ID). Cheia primară este compusă din cheile primare ale celor două tabele asociate.
- Tabela de asociere este folosită cel mai des pentru maparea asocierii n-la-n, dar poate fi folosită și pentru alte tipuri de asocieri (mai dificil, complex).

Tabela de asociere

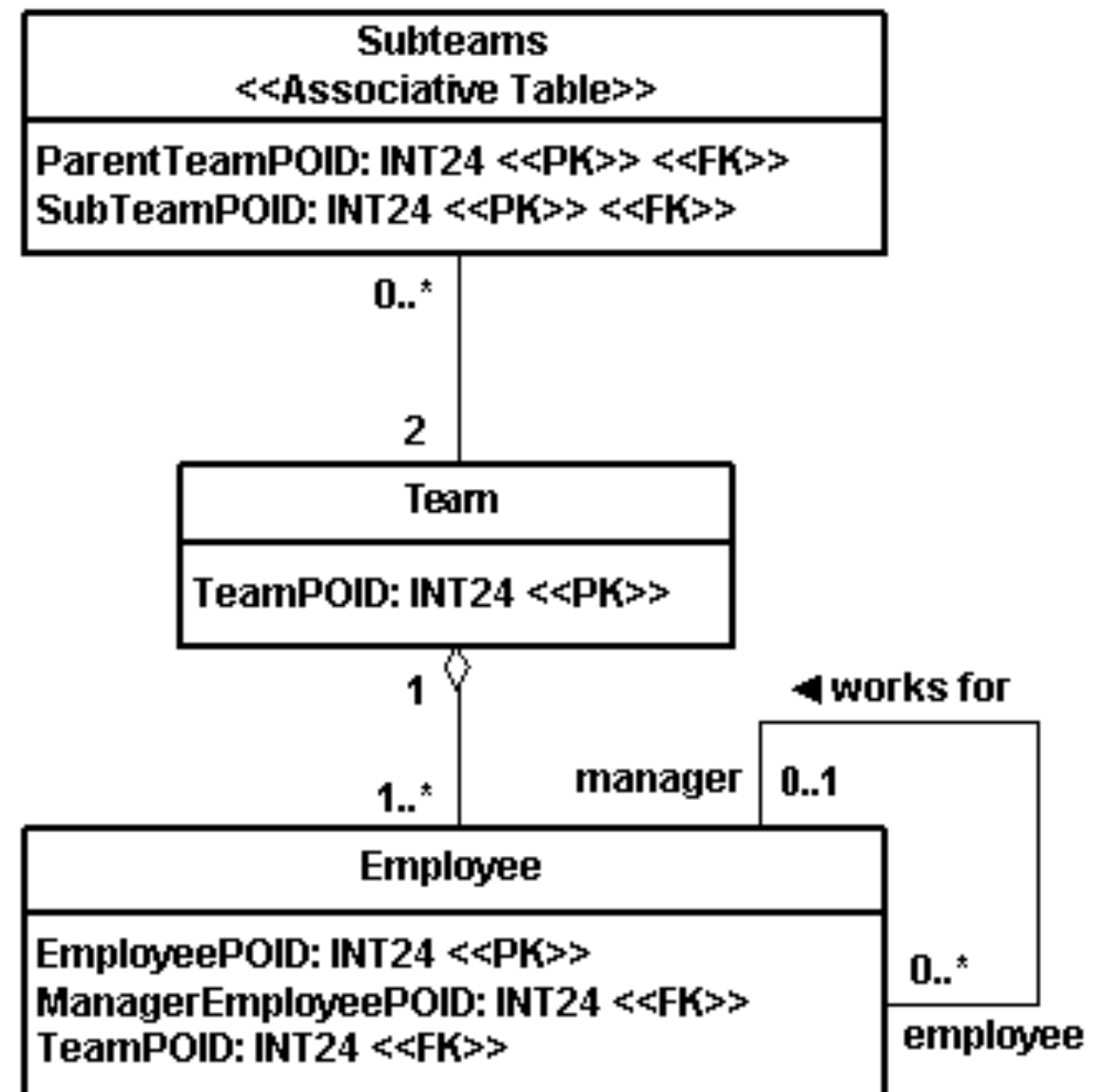


Maparea asocierilor recursive

<<Class Model>>



<<Physical Data Model>>



Maparea proprietăților statice

TableA

A
<u>StaticAttributeA</u> name

PK	Name	StaticAttributeA
	AA	1
	BB	2
	CC	1

TableA

PK	Name
	AA
	BB
	CC

StaticTableA

StaticAttributeA
1

Maparea proprietăților statice - Strategii (1)

- O tabelă cu o singură înregistrare, o singură coloană pentru fiecare proprietate statică:
 - Pro: Simplu, acces rapid
 - Con: multe tabele mici

A
<u>StaticAttributeA</u> name

B
<u>StaticAttrB1</u>
<u>StaticAttrB2</u>

C
<u>StaticAttrC1</u>

StaticTableA

StaticAttributeA
1

StaticTableB_1

StaticAttrB1
23

StaticTable_B2

StaticAttrB2
ValueA

StaticTableC

StaticAttrC1
23.56

Maparea proprietăților statice - Strategii (2)

- O tabelă cu mai multe coloane, o singură înregistrare pentru fiecare clasă:
 - Pro: Simplu, acces rapid
 - Con: multe tabele mici, dar mai puține decât la strategia precedentă

A
<u>StaticAttributeA</u> name

StaticTableA

StaticAttributeA
1

B
<u>StaticAttrB1</u> <u>StaticAttrB2</u>

StaticTableB

StaticAttrB1	StaticAttrB2
23	ValueA

C
<u>StaticAttrC1</u>

StaticTableC

StaticAttrC1
23.56

Maparea proprietăților statice - Strategii (3)

- O singură tabelă cu mai multe coloane - o singură înregistrare pentru toate clasele:
 - Pro: număr minim de tabele introdus
 - Con: potențiale probleme de concurență dacă mai multe clase trebuie să acceseze datele în același timp.

A
<u>StaticAttributeA</u> name

B
<u>StaticAttrB1</u> <u>StaticAttrB2</u>

C
<u>StaticAttrC1</u>

StaticAttributesTable

A.StaticAttributeA	B.StaticAttrB1	B.StaticAttrB2	C.StaticAttrC1
1	23	ValueA	23.56

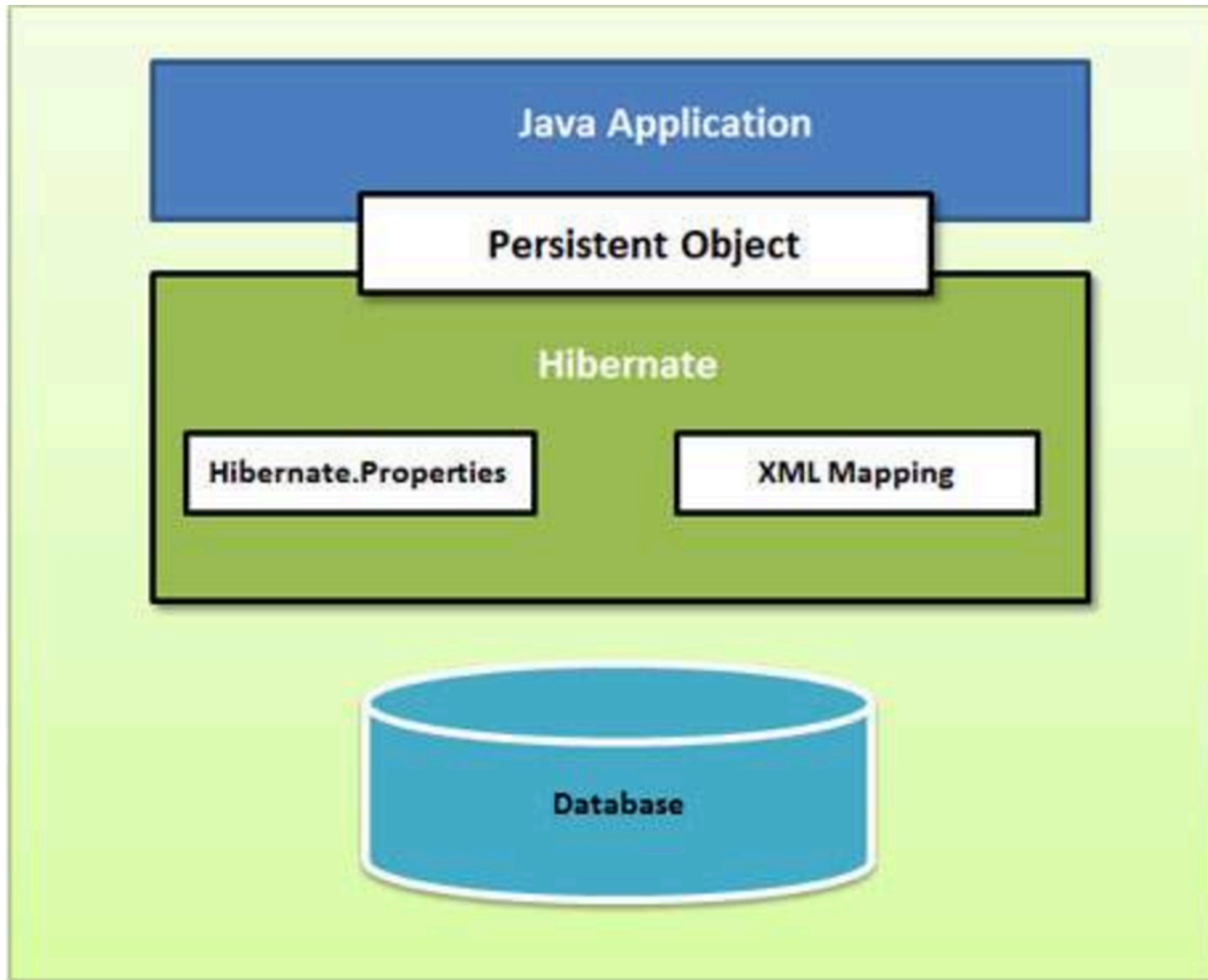
Maparea proprietăților statice

- Strategii (cont):
 - Schemă generică cu mai multe înregistrări pentru toate clasele:
 - Pro: număr minim de tabele introdus. Reduce problemele legate de concurență.
 - Con: Necesitatea convertirii între tipuri de date. Schema este asociată cu numele claselor și a proprietăților statice.

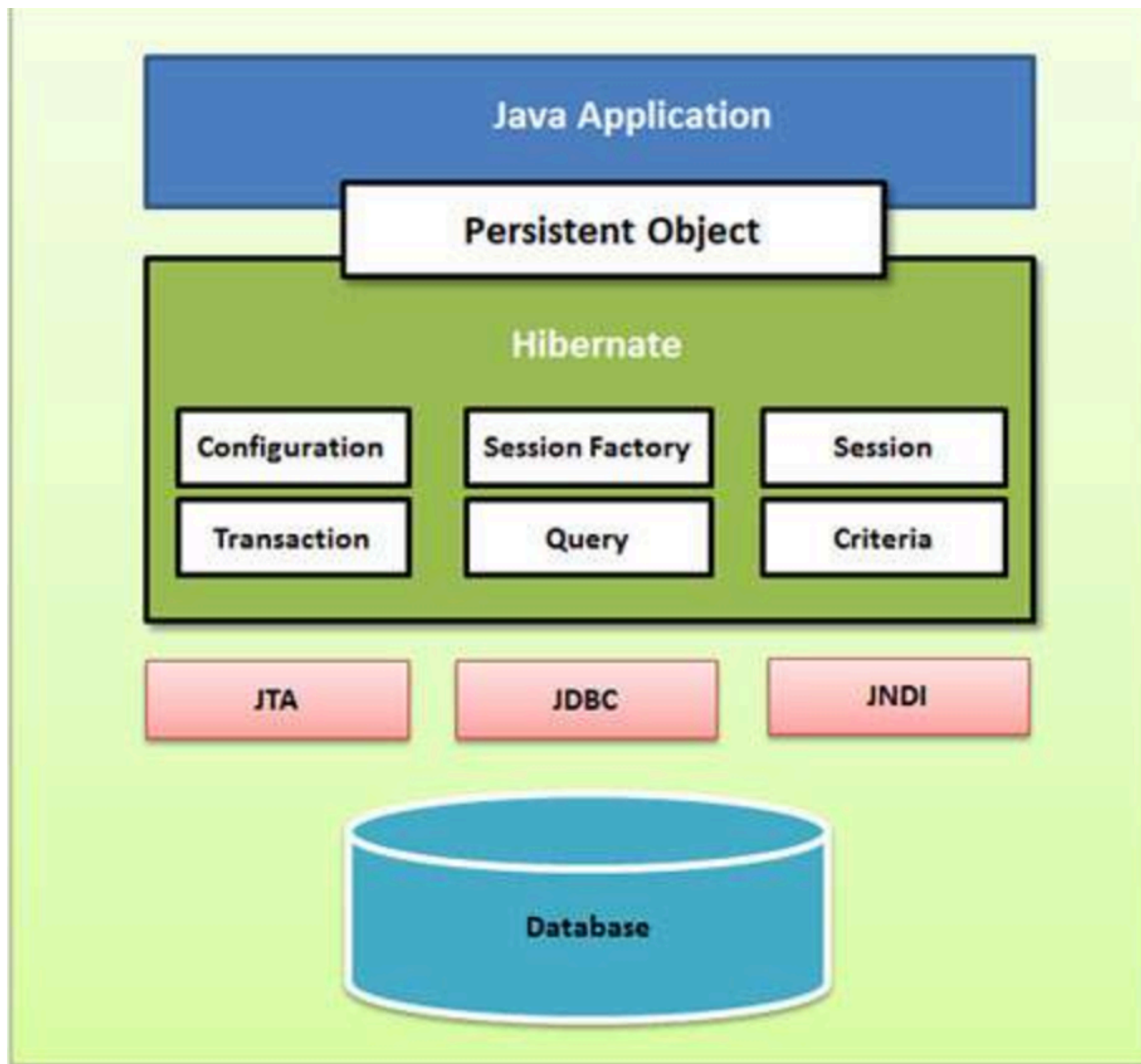
Hibernate

- Instrument open source pentru ORM.
- Aplicațiile care folosesc Hibernate definesc/specifică clasele persistente ce vor fi mapate în tabele într-o bază de date relațională.
- Toate instrucțiunile SQL sunt generate automat în timpul execuției.
- Informațiile despre mapare sunt transmise ca și un document de mapare în format XML sau prin adnotări.
- Documentul de mapare specifică:
 - cum vor fi mapate proprietățile la coloanele din tabelă (tabele).
 - strategia aleasă de dezvoltator pentru mapare (unde este cazul).

Arhitectura



Arhitectura



Arhitectura

- Interfețele/Clasele definite de Hibernate pot fi clasificate astfel:
 - Interfețe folosite de aplicații pentru a efectua operații CRUD și interogări. Logica aplicației corespunzătoare nivelului de persistență va fi strâns cuplată de acestea: *Session*, *Transaction* și *Query*.
 - Clase apelate de aplicație pentru configurarea instrumentului Hibernate: clasa *Configuration*.
 - Interfețe callback care permit aplicațiilor să reacționeze la evenimente ce apar în timpul execuției instrumentului Hibernate: *Interceptor*, *Lifecycle* și *Validatable*.
 - Interfețe care permit extinderea Hibernate: *UserType*, *CompositeUserType* și *IdentifierGenerator*.
- Hibernate folosește Java API existent: JDBC, Java Transaction API (JTA) și Java Naming and Directory Interface (JNDI).

Interfețe de bază

- Interfața *Session*: este cea mai folosită interfață de către aplicațiile care folosesc Hibernate. O instanță de tip *Session* nu consumă multe resurse și poate fi ușor creată/distrusă.
- Interfața *SessionFactory*. Aplicațiile obțin instanțe de tip *Session* folosind un obiect de tip *SessionFactory*. Păstrează în cache instrucțiunile SQL generate dinamic și alte metadate legate de mapări folosite în timpul execuției.
- Clasa *Configuration*. Un obiect de tip *Configuration* este folosit pentru configurarea și pornirea Hibernate. Aplicațiile pot folosi acest obiect pentru a specifica locația bazei de date și a altor proprietăți specifice Hibernate.
- Interfața *Transaction*. Abstractizează codul corespunzător unei tranzacții de implementarea specifică folosită: o tranzacție JDBC, o tranzacție JTA UserTransaction, etc.
- Interfața *Query*. Permite efectuarea de interogări asupra bazei de date și controlează modul în care este executată interogarea. Interogările pot fi scrise în HQL sau folosind direct limbajul SQL corespunzător bazei de date.

Exemplu

- Etape:
 - Crearea claselor Java corespunzătoare entităților
 - Adăugarea informațiilor pentru mapare (adnotări)
 - Crearea fișierului de configurare Hibernate
 - Implementarea nivelului de persistență folosind funcțiile corespunzătoare
 - Testarea claselor

Exemplu - Entitatea

@Entity

```
public class Book {  
    private Long id;  
    private String title;  
  
    public Book() {} //obligatoriu  
    public Book(String title){ this.title=title;}  
  
    @Id  
    @GeneratedValue(strategy=IDENTITY)  
    public Long getId() { return id;}  
  
    public void setId(Long id) {this.id = id;}  
  
    @NotNull  
    public String getTitle() { return title;}  
  
    public void setTitle(String title) {this.title = title;}  
}
```

Exemplu - Entitatea

```
@Entity
@Table( name = "Messages")
public class Message {
    @Id
    @Column(name = "idMessage")
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "Message_Text")
    private String text;

    public Message() {} //obligatoriu
    public Message(String text) { this.text = text; }

    public Long getId() { return id; }
    private void setId(Long id) { this.id = id; }

    public String getText() { return text; }
    public void setText(String text) { this.text = text; }
}
```

Fișierul de configurare *hibernate.properties*

```
#hibernate.properties –numele implicit căutat de Hibernate
```

```
# URL-ul de conectare la baza de date (JDBC)
```

```
jakarta.persistence.jdbc.url=jdbc:sqlite:/Users/grigo/HibernateExample/hibernate6.db
```

```
# Credentials
```

```
#jakarta.persistence.jdbc.user=
```

```
#jakarta.persistence.jdbc.password=
```

```
jakarta.persistence.schema-generation.database.action=update
```

```
#none/create/etc
```

```
# SQL statement logging
```

```
hibernate.show_sql=true
```

```
hibernate.format_sql=true
```

```
hibernate.highlight_sql=true
```

Exemplu – Salvarea unei entități

```
//INSERT
```

```
void addBook(Book b) {  
    sessionFactory.inTransaction(session -> {  
        session.persist(b);  
        System.out.println("Book added "+b);  
    });  
}
```

```
//Book b=new Book("Test Book ");
```

```
//dupa rulare
```

```
//Book added Book{id='8', title='Test Book '}
```